DevOps Engineering • Container Series

Docker

Complete Interview Reference

30	6	3	Intermediate
Questions	Sections	Diff. Levels	DevOps Level

by Sandhya
DevOps Engineer

ABOUT THIS GUIDE

30 comprehensive questions covering Docker architecture, networking, volumes, Compose, security, and real-world production troubleshooting.

Sections: Docker Fundamentals • Networking • Volumes & Storage • Docker Compose • Security & Best Practices • Real-World & Advanced

Every answer goes DEEP — architecture internals, real Dockerfile examples, production patterns, and the WHY behind every concept.

Difficulty: Easy (green) | Medium (yellow) | Hard (red)

TABLE OF CONTENTS

SECTION 01

Docker Fundamentals

Architecture, images, containers, Dockerfiles and layer caching

IN THIS SECTION

Q01 What is Docker? How is it different from a Virtual Machine? Explain with architecture.

Easy

▸ **DETAILED ANSWER**

Docker is a containerization platform that packages an application and ALL its dependencies (code, runtime, libraries, config) into a single portable unit called a container. The container runs identically on any machine that has Docker installed — no more 'works on my machine' problems.

Architecture Comparison:

Layer	Virtual Machine (VM)	Docker Container
Hardware	Physical server	Physical server
Hypervisor	Type 1 (bare metal) or Type 2 (on OS)	Not needed
Guest OS	Full OS per VM (Windows/Linux — GBs)	NO OS — shares host kernel
Runtime	Application + full OS stack	Application + minimal libs only
Size	GBs — full OS included	MBs — only what app needs
Startup time	Minutes — boot full OS	Seconds or milliseconds
Isolation	Hardware-level — full isolation	Process-level — kernel namespaces
Resource usage	Heavy — full OS per VM	Lightweight — shared kernel

How Docker achieves isolation WITHOUT a separate OS:

Docker uses TWO Linux kernel features to create container isolation:

→ Namespaces: Give each container its own view of the system — PID namespace (own process tree), NET namespace (own network stack), MNT namespace (own filesystem mount points), UTS (own hostname).

→ cgroups (Control Groups): Limit and account for resource usage — CPU, memory, disk I/O. This prevents one container from consuming all host resources.

Docker Architecture components:

Component	Role
Docker Client (CLI)	What you interact with: docker run, docker build, docker push
Docker Daemon (dockerd)	Background service that manages containers, images, volumes, networks
Docker Registry	Storage for images: Docker Hub (public), ECR/GCR (private cloud)
containerd	Low-level container runtime that actually creates and runs containers
runc	OCI-compliant runtime that interfaces with Linux kernel namespaces/cgroups

INTERVIEW TIP

The key insight: VMs virtualize HARDWARE (each VM thinks it has its own CPU, RAM, disk). Docker virtualizes the OS (each container thinks it has its own filesystem and processes but shares the SAME kernel). This is why Docker is so much faster and lighter.

KEY TAKEAWAYS

Docker = process isolation via Linux namespaces + cgroups. Shares host kernel. No guest OS.
VM = full hardware virtualization. Separate OS per VM. Heavy but stronger isolation.
Container startup: milliseconds. VM startup: minutes. Container size: MBs. VM size: GBs.
Docker Client talks to Docker Daemon (dockerd) via REST API. Daemon does the real work.

Q02 What is a Docker Image vs a Docker Container? What is the relationship between them?

Easy

▸ DETAILED ANSWER

This is the most fundamental Docker concept. Getting this crystal clear is non-negotiable for any interview.

Aspect	Docker Image	Docker Container
Definition	Read-only template/blueprint for creating containers	Running instance of an image — has live state
State	Static — immutable, never changes	Dynamic — running, stopped, paused

Storage	Stored in layers on disk / registry	Thin writable layer on top of image layers
Analogy	A class definition in OOP / a cookie cutter	An object instance / a cookie made from cutter
Created by	docker build (from Dockerfile)	docker run (from an image)
Multiple instances?	One image → infinite containers	Each container is independent
Modification	Cannot modify — must rebuild image	Can write files but changes lost on removal
Sharing	Push to registry — anyone can pull	Cannot share a running container directly

The relationship in practice:

```
# 1. Build an image from Dockerfile:
docker build -t myapp:v1.0 .

# 2. List images (blueprints on disk):
docker images
# REPOSITORY  TAG     IMAGE ID      SIZE
# myapp       v1.0    a1b2c3d4e5f6  245MB

# 3. Run a container FROM the image (spawn an instance):
docker run -d --name app1 myapp:v1.0
docker run -d --name app2 myapp:v1.0   # second container, same image
docker run -d --name app3 myapp:v1.0   # third container, same image

# 4. List containers (running instances):
docker ps
# CONTAINER ID  IMAGE       NAMES
# a1b2c3       myapp:v1.0  app1
# b2c3d4       myapp:v1.0  app2   <- same image, different container
# c3d4e5       myapp:v1.0  app3
```

The Copy-on-Write (CoW) layer:

When you run a container, Docker does NOT copy the entire image. Instead it adds a thin WRITABLE layer on top of the read-only image layers. Any files you write inside the container go to this writable layer. When the container is deleted, the writable layer is gone — the image remains untouched. This is why containers are fast to create and lightweight.

INTERVIEW TIP

Analogy that always lands in interviews: Image is like a CLASS in object-oriented programming. Container is like an OBJECT (instance) of that class. You can create 100 objects from one class — same principle. The class doesn't change when objects are created or destroyed.

KEY TAKEAWAYS

Image = immutable blueprint (read-only). Container = running instance (has writable layer).

1 image → unlimited containers. All share the same read-only image layers.

Container writes go to a thin CoW layer on top — deleted when container is removed.
docker build → creates image. docker run → creates container FROM image.

Q03 What is a Dockerfile? Explain each instruction — FROM, RUN, COPY, ADD, CMD, ENTRYPOINT, ENV, EXPOSE, WORKDIR.

Medium

▸ **DETAILED ANSWER**

A Dockerfile is a text file with instructions that Docker reads top-to-bottom to build an image. Every instruction creates a new layer in the image. Getting Dockerfile syntax right is the foundation of everything else in Docker.

Instruction	Purpose	Key Points
FROM	Base image to build FROM	First instruction always. Every image starts from a base.
RUN	Execute command during BUILD time	Creates a new layer. Use for installing packages.
COPY	Copy files from host to image	Preferred over ADD. Simple and explicit.
ADD	Like COPY but also extracts tarballs and fetches URLs	Use only when you need tar extraction. COPY otherwise.
CMD	Default command when container STARTS	Can be overridden by docker run <command>. Only one CMD.
ENTRYPOINT	Fixed command that ALWAYS runs	Cannot be overridden. Use for container's main purpose.
ENV	Set environment variables	Available at build AND runtime. Use for config.
EXPOSE	Document which port the app uses	Informational only — does NOT actually publish the port.
WORKDIR	Set working directory	Like 'cd' but persistent. Creates dir if it doesn't exist.
USER	Switch to non-root user	Security best practice — run app as non-root.
ARG	Build-time variable	Only available during build. Not in running container.
VOLUME	Declare a mount point	Documents where container expects persistent data.

Production-grade Dockerfile — Node.js Application:

```
# Stage 1: Builder
FROM node:20-alpine AS builder
```

```
# Set working directory
WORKDIR /app

# Copy package files FIRST (layer cache optimization)
COPY package*.json ./

# Install dependencies
RUN npm ci --only=production

# ——————————————————————————
# Stage 2: Production image
FROM node:20-alpine

# Security: create non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

WORKDIR /app

# Copy only built artifacts from builder stage
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json .

# Copy application source
COPY src/ ./src/

# Environment variables
ENV NODE_ENV=production
ENV PORT=3000

# Document exposed port (informational)
EXPOSE 3000

# Switch to non-root user
USER appuser

# ENTRYPOINT = always runs. CMD = default args.
ENTRYPOINT ["node"]
CMD ["src/index.js"]
```

INTERVIEW TIP

The order of instructions in a Dockerfile matters for CACHING. Always copy package.json and run npm install BEFORE copying source code. Why? Package dependencies change rarely; source code changes constantly. If you copy source first, every code change invalidates the npm install layer — very slow builds.

KEY TAKEAWAYS

FROM = base image. RUN = build-time commands (installs). COPY = bring files in.
WORKDIR = set working directory. Persists for all subsequent instructions.
EXPOSE = documentation only. Does NOT publish ports — that's done with docker run -p.
ARG = build-time only. ENV = build AND runtime. Use ARG for build secrets, ENV for app config.

Q04 What is the difference between CMD and ENTRYPOINT in a Dockerfile? When would you use each?

Hard

▸ DETAILED ANSWER

CMD vs ENTRYPOINT is the most commonly misunderstood Docker concept. The difference is subtle but crucial for building properly behaving containers.

Aspect	CMD	ENTRYPOINT
Purpose	Default command/args if none provided	The fixed executable the container ALWAYS runs
Override	Can be completely overridden: docker run myimg <cmd>	Cannot override without --entrypoint flag
Used alone	docker run myimg → runs CMD	docker run myimg → runs ENTRYPOINT
Used together	CMD becomes DEFAULT ARGS to ENTRYPOINT	ENTRYPOINT is the executable, CMD provides args
Syntax (exec)	CMD ["node", "app.js"]	ENTRYPOINT ["node"]
Syntax (shell)	CMD node app.js	ENTRYPOINT node app.js
Best for	Default behavior that users might want to change	Containers designed to run a specific program

The POWERFUL combination — ENTRYPOINT + CMD together:

```
# Dockerfile:
ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8080"]          # default args

# docker run myimage
# → runs: python app.py --port 8080  (uses CMD default args)

# docker run myimage --port 9090
# → runs: python app.py --port 9090  (CMD overridden, ENTRYPOINT fixed)

# docker run myimage --help
# → runs: python app.py --help       (perfect for CLI tools!)

# ──────────────────────────────────────────────────────────
# CMD alone:
CMD ["nginx", "-g", "daemon off;"]

# docker run myimage
# → runs: nginx -g daemon off;

# docker run myimage bash
# → runs: bash  (CMD completely replaced — useful for debugging!)
```

Exec form vs Shell form — always use exec form:

```
# Shell form (BAD for signals):
CMD node app.js
# → Actually runs: /bin/sh -c 'node app.js'
# → node is NOT PID 1. Shell is PID 1.
# → docker stop sends SIGTERM to shell, NOT to node
# → node doesn't get signal → container takes 10s to force kill

# Exec form (CORRECT):
CMD ["node", "app.js"]
# → node IS PID 1 directly
# → docker stop sends SIGTERM directly to node
# → node handles graceful shutdown immediately
```

INTERVIEW TIP

Always use exec form [ ] not shell form for CMD and ENTRYPOINT. Shell form wraps the command in /bin/sh -c which means your process is NOT PID 1 and won't receive Unix signals properly. This causes containers to take 10 seconds to stop because Docker has to force-kill them after timeout.

KEY TAKEAWAYS

CMD = overridable default command. ENTRYPOINT = always-runs fixed executable.
ENTRYPOINT + CMD together: ENTRYPOINT is the program, CMD is the default arguments.
docker run myimage <args> overrides CMD but NOT ENTRYPOINT.
Always use exec form ["cmd", "arg"] not shell form — ensures correct PID 1 and signal handling.

Q05 What is Docker Hub? What is the difference between a public and private registry?

Easy

DETAILED ANSWER

A Docker registry is a storage and distribution system for Docker images. When you run 'docker pull nginx', Docker fetches the nginx image from a registry. Understanding registries is critical for CI/CD pipelines.

Registry	Type	Best Used For
Docker Hub	Public cloud registry	Open source projects, public images, learning
AWS ECR	Private cloud (AWS)	Production workloads on AWS ECS/EKS
Google GCR / GAR	Private cloud (GCP)	Production workloads on GKE
Azure ACR	Private cloud (Azure)	Production workloads on AKS
GitHub Container Registry	Private + public	Open source projects with GitHub CI/CD
Harbor	Self-hosted private	On-premise, air-gapped environments

JFrog Artifactory	Enterprise private	Large orgs with multiple artifact types

Image naming convention — understanding the full image name:

```
# Full image name format:
# [registry]/[namespace]/[repository]:[tag]

nginx                          # short form = docker.io/library/nginx:latest
nginx:1.25                     # specific version tag
sandhya/myapp:v2.1             # Docker Hub personal namespace
123456.dkr.ecr.us-east-1.amazonaws.com/myapp:prod  # AWS ECR
gcr.io/my-project/myapp:latest # Google Container Registry

# Working with private registry:
docker login 123456.dkr.ecr.us-east-1.amazonaws.com
docker tag myapp:v1 123456.dkr.ecr.us-east-1.amazonaws.com/myapp:v1
docker push 123456.dkr.ecr.us-east-1.amazonaws.com/myapp:v1
docker pull 123456.dkr.ecr.us-east-1.amazonaws.com/myapp:v1
```

COMMON MISTAKES

Never push production images to public Docker Hub — your code, config, and potentially secrets become public.
Never use :latest tag in production — it's mutable (can change anytime). Always use specific version tags.
Never store secrets (API keys, passwords) in Docker images — they are visible via docker history.

KEY TAKEAWAYS

Docker Hub = default public registry. docker.io/library/nginx = official nginx image.
Production = always use private registry (ECR, GCR, ACR) for your own images.
Image name format: [registry]/[namespace]/[repo]:[tag]. No registry = defaults to Docker Hub.
Use specific version tags in production. NEVER :latest — it breaks reproducible deployments.

Q06 What are Docker layers? How does layer caching work and how do you optimize builds?

Hard

▸ DETAILED ANSWER

Docker images are built in layers — each instruction in a Dockerfile creates one layer. Layers are the secret to Docker's efficiency. Understanding them is what separates good Docker usage from great Docker usage.

How layers work:

Each layer is an immutable filesystem diff — it contains only the changes made by that instruction. Layers are identified by a SHA256 hash of their content. If the content doesn't change, the hash doesn't change, so Docker can REUSE the cached layer instead of rebuilding it.

```
# Build and see layers being created:
```

```
docker build -t myapp .
# Step 1/6 : FROM node:20-alpine     → pulls/uses cached layer
# Step 2/6 : WORKDIR /app            → new layer (tiny)
# Step 3/6 : COPY package*.json ./   → new layer
# Step 4/6 : RUN npm ci              → new layer (BIG - downloads packages)
# Step 5/6 : COPY src/ .             → new layer
# Step 6/6 : CMD ["node","app.js"] → new layer (tiny)

# View layers of an image:
docker history myapp
docker inspect myapp | grep -A 20 'Layers'
```

Layer Cache Invalidation — the golden rule:

When any layer changes, ALL SUBSEQUENT LAYERS are invalidated and must be rebuilt. This is why instruction ORDER matters enormously.

```
# BAD — source code change invalidates npm install every time:
COPY . .              # copies everything including source code
RUN npm ci            # ALWAYS re-runs because COPY above always changes

# GOOD — npm install cached unless package.json changes:
COPY package*.json ./ # copy ONLY package files first
RUN npm ci            # only re-runs when package.json changes
COPY . .              # source code copied AFTER npm install

# Result: 99% of builds skip the npm install step = massively faster CI/CD
```

Layer optimization strategies:

Strategy	How to Apply
Copy dependencies first	COPY package.json → RUN install → COPY source code
Combine RUN commands	RUN apt-get update && apt-get install -y pkg1 pkg2 (one layer not two)
Clean up in same RUN	RUN apt-get install pkg && apt-get clean && rm -rf /var/lib/apt/lists/*
Use .dockerignore	Exclude node_modules, .git, logs from COPY context
Use slim base images	node:20-alpine vs node:20 — alpine saves ~700MB
Multi-stage builds	Compile/build in stage 1, only copy artifacts to stage 2

INTERVIEW TIP

Combining RUN commands with && is not just style — it's functional. If you do RUN apt-get update in layer 5 and RUN apt-get install pkg in layer 6, a rebuild might use cached layer 5 (stale apt cache) but run layer 6 against that stale cache. Combine them: RUN apt-get update && apt-get install -y pkg to guarantee they always run together.

KEY TAKEAWAYS

Every Dockerfile instruction = one image layer. Layers are cached by content hash.

Cache invalidation rule: changed layer = ALL layers after it are rebuilt.
Optimize: copy slow-changing files first (dependencies), fast-changing files last (source).
Combine RUN commands with && to reduce layers and avoid stale cache issues.

SECTION 02

Docker Networking

Bridge, host, overlay networks, DNS, and container communication

IN THIS SECTION

Q07 What are the different Docker network types? Explain bridge, host, none, and overlay.

Medium

▶ DETAILED ANSWER

Network Type	How It Works	Use Case
bridge (default)	Creates virtual switch on host. Containers get own IP in 172.17.0.0/16 subnet.	Single-host container communication. Default for docker run.
host	Container shares host's network stack. No isolation. Container port = host port.	Maximum performance. No port mapping needed. Linux only.
none	No networking at all. Completely isolated.	Maximum security. Batch jobs that don't need network.
overlay	Spans multiple Docker hosts. Used in Swarm.	Multi-host container communication in Swarm clusters.
macvlan	Container gets its own MAC address on physical network.	Legacy apps that expect direct network access.
Custom bridge	User-defined bridge with automatic DNS between containers.	ALWAYS use instead of default bridge for production.

```
# List all networks:
docker network ls

# Create a custom bridge network:
docker network create --driver bridge myapp-network
```

```
# Run containers on custom network:
docker run -d --name db --network myapp-network postgres:15
docker run -d --name api --network myapp-network myapi:v1

# Inspect network (see connected containers and IPs):
docker network inspect myapp-network

# Connect existing container to network:
docker network connect myapp-network mycontainer

# Host network mode — container uses host's network directly:
docker run -d --network host nginx
# nginx listens on port 80 of the HOST directly — no -p needed
```

INTERVIEW TIP

ALWAYS create a custom bridge network instead of using the default bridge. The default bridge does NOT support automatic DNS — containers can't reach each other by name, only by IP. Custom bridge networks automatically provide DNS, so containers can communicate using container names. This is the #1 Docker networking mistake.

KEY TAKEAWAYS

Default bridge: automatic. Containers get IPs but NO automatic DNS between them.
Custom bridge: creates automatic DNS. Containers find each other by NAME. Always use this.
Host network: shares host network stack. Best performance but zero network isolation.
none: completely isolated. No network access. For security-sensitive batch workloads.

Q08 How do containers communicate with each other? What is the difference between --link and Docker networks?

Medium

DETAILED ANSWER

Container-to-container communication has evolved significantly. The old --link flag is legacy. Modern Docker uses user-defined networks. Understanding both helps explain why networks are superior.

Aspect	--link (Legacy)	Custom Network (Modern)
DNS resolution	Manually injects /etc/hosts entry	Automatic DNS for ALL containers on network
Direction	One-way: source can reach target	Both ways: any container can reach any other
Dynamic?	Static — if target restarts with new IP, link breaks	Dynamic — DNS always resolves current IP
Scope	Only between specifically linked pairs	All containers on same network automatically
Current status	Deprecated — may be removed	Official recommended approach
Compose usage	services: depends_on (old link style)	networks: section in docker-compose.yml

Modern approach — custom networks (the right way):

```
# Create network:
docker network create app-net

# Start database container:
docker run -d \
  --name postgres-db \
  --network app-net \
  -e POSTGRES_PASSWORD=secret \
  postgres:15

# Start API — connects to DB by NAME 'postgres-db':
docker run -d \
  --name api-service \
  --network app-net \
  -e DB_HOST=postgres-db \
  -e DB_PORT=5432 \
  myapi:latest

# Inside api-service, you can now reach database via:
# hostname: postgres-db  (Docker DNS resolves this automatically)
# ping postgres-db → resolves to container's IP
```

INTERVIEW TIP

In Docker Compose, all services in the same compose file are automatically on a custom bridge network — that's why services can reach each other by service name. The 'web' service can connect to 'db:5432' because Compose creates a network and handles DNS automatically. This is why Compose is so powerful for local development.

KEY TAKEAWAYS

--link is deprecated. Do not use in new projects.
Custom bridge networks provide automatic DNS — containers reach each other by container name.
docker network create + --network flag = modern, reliable container networking.
Docker Compose auto-creates a custom network for all services in the same file.

Q09 What is port mapping in Docker? Explain the -p and -P flags with examples.

Easy

▸ DETAILED ANSWER

Port mapping is how you make a containerized application accessible from outside the container. By default, container ports are completely isolated — nothing can reach them from the host or external network.

Flag	Syntax	Behavior
-p (publish)	docker run -p HOST:CONTAINER	Map specific host port to container port. You choose the ports.

-p (any host)	docker run -p 0.0.0.0:8080:80	Bind to all interfaces on host port 8080.
-p (random)	docker run -p 80	Docker picks a random available host port for container port 80.
-P (uppercase)	docker run -P	Maps ALL EXPOSE'd ports to random host ports automatically.
No flag	docker run (no -p)	Port is accessible ONLY within Docker networks. Not from host.

```
# Map host port 8080 to container port 80:
docker run -d -p 8080:80 nginx
# Access: http://localhost:8080 → nginx inside container on port 80

# Map multiple ports:
docker run -d -p 8080:80 -p 8443:443 nginx

# Bind only to specific interface (security):
docker run -d -p 127.0.0.1:5432:5432 postgres
# Only accessible from localhost — NOT from external network

# Random host port (-P uppercase):
docker run -d -P nginx
docker port <container_id>    # shows: 80/tcp -> 0.0.0.0:32768

# Check port mappings of running container:
docker port mycontainer
docker inspect mycontainer | grep -A 10 'PortBindings'
```

EXPOSE vs -p — critical distinction:

EXPOSE in Dockerfile is documentation — it tells other developers which port the app listens on. It does NOT actually open or publish the port. The -p flag in docker run is what actually makes the port accessible from outside.

INTERVIEW TIP

In production, always bind sensitive ports (databases) to 127.0.0.1 only: -p 127.0.0.1:5432:5432. Without this, the port is bound to 0.0.0.0 and accessible from ANY network interface on the host — a security risk if the host is internet-facing.

KEY TAKEAWAYS

-p HOST:CONTAINER maps host port to container port. -P maps all EXPOSE'd ports randomly.
EXPOSE in Dockerfile = documentation only. -p in docker run = actual port publishing.
Bind to 127.0.0.1 for local-only access: -p 127.0.0.1:5432:5432 (databases, admin panels).
Containers on same Docker network communicate directly — no port mapping needed between them.

Q10 What is Docker DNS? How do containers resolve each other by name inside a custom network?

Medium

▸ **DETAILED ANSWER**

Docker's embedded DNS server is one of its most powerful features for microservices. It allows containers to discover and communicate with each other using simple hostnames instead of brittle IP addresses.

How Docker DNS works:

Every container on a user-defined network is automatically registered with Docker's embedded DNS server at 127.0.0.11. When container A tries to resolve 'container-b', the DNS query goes to 127.0.0.11, which looks up the container name and returns its current IP address.

```
# Create network and start containers:
docker network create microservices
docker run -d --name redis-cache  --network microservices redis:7
docker run -d --name postgres-db  --network microservices postgres:15
docker run -d --name api-gateway  --network microservices mygateway

# Inside api-gateway container, DNS resolution works automatically:
docker exec -it api-gateway sh

# These all work — resolved by Docker DNS:
ping redis-cache     # resolves to redis container IP
ping postgres-db     # resolves to postgres container IP
curl http://redis-cache:6379   # Redis connection by name

# Docker DNS server is always at 127.0.0.11:
cat /etc/resolv.conf
# nameserver 127.0.0.11
# options ndots:0

# DNS also works for container ALIASES:
docker network connect --alias db microservices postgres-db
# now 'db' also resolves to postgres-db
```

DNS load balancing with Docker Swarm:

In Docker Swarm, when you have 5 replicas of a service named 'api', the service name 'api' resolves via VIP (Virtual IP) to all 5 replicas with built-in load balancing. DNS returns the VIP, and the Swarm routing mesh distributes traffic across replicas.

INTERVIEW TIP

In microservices architecture, your application config should use SERVICE NAMES not IPs. For example: DB_HOST=postgres-db not DB_HOST=172.17.0.3. IPs change when containers restart. Names are stable — they always resolve to the current container, even after restarts.

KEY TAKEAWAYS

Docker DNS runs at 127.0.0.11 inside every container on a user-defined network.
Container name = hostname. Any container on same network can reach another by name.
Default bridge network: NO automatic DNS. Custom bridge: automatic DNS. Always use custom.
In microservices: configure app connections using container/service names, never IP addresses.

Q11 What is an overlay network in Docker Swarm? When would you use it? — Hard

▸ **DETAILED ANSWER**

An overlay network is Docker's solution for connecting containers across MULTIPLE physical hosts. Bridge networks only work on a single host. Overlay networks create a virtual Layer 2 network that spans the entire Swarm cluster.

How overlay networking works:

Docker uses VXLAN (Virtual Extensible LAN) to encapsulate container traffic inside UDP packets that travel over the existing host network. From the container's perspective, it thinks it's on a flat Layer 2 network — it doesn't know or care that the other container is on a different physical machine.

Aspect	Bridge Network	Overlay Network
Scope	Single Docker host only	Spans entire Swarm cluster
Use case	Single-host development / docker-compose	Production Swarm multi-host deployments
Technology	Linux veth pairs + iptables	VXLAN encapsulation over UDP port 4789
Requires	Just Docker daemon	Docker Swarm mode initialized
Container communication	By name on same host	By service name across any host in Swarm
Ports required	None	TCP 2377 (Swarm mgmt), UDP 4789 (VXLAN)

```
# Initialize Swarm (first manager):
docker swarm init --advertise-addr 192.168.1.10

# Create overlay network:
docker network create --driver overlay --attachable myoverlay

# Deploy service on overlay network (across all nodes):
docker service create \
  --name api \
  --network myoverlay \
  --replicas 3 \
  myapi:latest

# All 3 replicas (possibly on different hosts) can communicate
# via the overlay network as if they're on the same LAN

# Check overlay networks:
docker network ls --filter driver=overlay
```

INTERVIEW TIP

In modern Kubernetes-dominated environments, overlay networks are primarily relevant for Docker Swarm. However, the CONCEPT is identical to what Kubernetes uses (Flannel, Calico, Weave are all overlay network implementations). Understanding Docker overlay deeply gives you a strong foundation for Kubernetes networking.

KEY TAKEAWAYS

Overlay = multi-host networking. Uses VXLAN to tunnel container traffic across hosts.

Requires Docker Swarm mode. Bridge network = single host. Overlay = entire Swarm cluster.

Services on same overlay network communicate by service name regardless of which host they're on.

Kubernetes uses the same overlay concept (CNI plugins: Flannel, Calico, Weave, Cilium).

SECTION 03

Volumes & Storage

Persisting data, sharing between containers, and storage best practices

IN THIS SECTION

Q12 What is the difference between a Docker Volume, a Bind Mount, and a tmpfs mount?

Medium

▶ DETAILED ANSWER

Aspect	Volume	Bind Mount	tmpfs Mount
Storage location	Docker-managed: /var/lib/docker/volumes/	Anywhere on host filesystem	RAM only — never touches disk
Managed by	Docker completely	You manage host path	Kernel manages in memory
Portability	Works on any Docker host	Host path must exist	Always portable (RAM)
Performance	Good — Docker optimized	Same as host filesystem	Excellent — in memory
Persists after rm?	YES — survives container removal	YES — host file independent	NO — lost on container stop
Backup/restore	docker volume commands	Standard OS file tools	Not applicable
Use case	Databases, app data	Dev: live code reload	Secrets, temp data, caches

```
# Docker Volume (recommended for production):
docker run -d \
  --name postgres \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15
# 'pgdata' is a named volume managed by Docker

# Bind Mount (great for development):
docker run -d \
```

```
  --name devapp \
  -v $(pwd)/src:/app/src \
  -v $(pwd)/config:/app/config \
  myapp:dev
# Edit files on host → changes immediately reflected in container

# tmpfs Mount (for sensitive data in memory):
docker run -d \
  --name secure-app \
  --tmpfs /run/secrets:rw,noexec,nosuid,size=64m \
  myapp:latest
# /run/secrets exists in RAM only — never written to disk
```

REAL-WORLD USE

Production databases: ALWAYS use named volumes for PostgreSQL/MySQL data — Docker manages backups and migration.

Development workflow: bind mount source code so code changes are instant without rebuilding image.

Docker secrets / session tokens: tmpfs so sensitive data never touches disk — critical for compliance.

INTERVIEW TIP

For databases in production, named volumes are superior to bind mounts. Volumes are managed by Docker, easy to backup with 'docker run --volumes-from', and portable across different host filesystem layouts. With bind mounts, you're responsible for directory permissions, ownership, and backups.

KEY TAKEAWAYS

Volume = Docker-managed. Best for production. Persists after container removed.

Bind Mount = host path. Best for development (live reload). Requires correct host setup.

tmpfs = RAM only. Best for secrets and sensitive temp data. Lost when container stops.

Production rule: databases → named volumes. Dev code → bind mounts. Secrets → tmpfs.

Q13 How do you persist data in Docker containers? What happens to data when a container is removed?

Easy

▸ DETAILED ANSWER

This is the most critical Docker concept for production deployments. By default, ALL data written inside a container is EPHEMERAL — it disappears forever when the container is removed. This is by design.

What happens to data on container removal:

Data Location	What Happens on docker rm	Survives?
Container's writable layer	DELETED permanently	✗ NO

Named volume (-v myvolume:/path)	Volume remains — data safe	✓✓ YES
Bind mount (-v /host/path:/path)	Host file unchanged — data safe	✓✓ YES
tmpfs mount	Already gone — lost on container stop	✗ NO
Database files in container (no volume)	DELETED permanently — catastrophic!	✗ NO

```
# DANGEROUS — data lost on container removal:
docker run -d --name mydb postgres:15
docker rm -f mydb   # ALL database data is gone forever!

# CORRECT — data persists in named volume:
docker run -d --name mydb -v pgdata:/var/lib/postgresql/data postgres:15
docker rm -f mydb   # container gone but 'pgdata' volume still exists
docker run -d --name mydb2 -v pgdata:/var/lib/postgresql/data postgres:15
# mydb2 has ALL data from mydb — volume reconnected

# Managing volumes:
docker volume ls                    # list all volumes
docker volume inspect pgdata        # volume details and mount path
docker volume rm pgdata             # remove volume (only if no container uses it)
docker volume prune                 # remove ALL unused volumes (DANGEROUS)

# Backup a volume:
docker run --rm \
  -v pgdata:/source:ro \
  -v $(pwd):/backup \
  alpine tar czf /backup/pgdata-backup.tar.gz -C /source .
```

COMMON MISTAKES

Running a database container WITHOUT a volume is a production disaster waiting to happen.
docker volume prune removes ALL unused volumes — never run this in production without careful review.
docker rm -v removes container AND its anonymous volumes. Use named volumes to avoid accidental deletion.

✓✓ KEY TAKEAWAYS

Container writable layer = ephemeral. Deleted permanently when container is removed.
Use named volumes for ANY data that must survive container lifecycle (databases, uploads).
Volume survives docker rm. Volume only deleted with docker volume rm or docker volume prune.
Back up volumes by mounting them into a temporary container and using tar/rsync.

Q14 What is a named volume vs anonymous volume? How do you manage volumes with docker volume commands?

Medium

▶ DETAILED ANSWER

Aspect	Named Volume	Anonymous Volume
Creation	docker volume create myvol OR -v myname:/path	Docker auto-generates a random UUID name
Name	Human readable: pgdata, app-uploads	Random hash: a1b2c3d4e5f6...
Reuse	Easy — reference by name across containers	Hard — must find UUID to reuse
docker rm -v	NOT removed (explicitly named — safe)	REMOVED if you use docker rm -v flag
Best for	Production data, databases, explicit sharing	Temporary container-only data
Lifecycle	You control when it's deleted	Tied to container lifecycle

```
# Create named volume explicitly:
docker volume create pgdata
docker volume create app-uploads

# Use named volume:
docker run -d -v pgdata:/var/lib/postgresql/data postgres:15

# Anonymous volume (avoid in production):
docker run -d -v /var/lib/postgresql/data postgres:15
# Docker creates: a1b2c3d4e5f67890.../  (UUID name)

# Full docker volume command reference:
docker volume ls                          # list all volumes
docker volume ls -f dangling=true         # list unused/orphaned volumes
docker volume inspect pgdata              # full details of a volume
docker volume rm pgdata                   # delete specific volume
docker volume prune                       # delete ALL unused volumes
docker volume prune -f                    # skip confirmation prompt

# Find where Docker stores volume data on host:
docker volume inspect pgdata | grep Mountpoint
# 'Mountpoint': '/var/lib/docker/volumes/pgdata/_data'
```

INTERVIEW TIP

In production, always use NAMED volumes and never anonymous volumes. Named volumes are self-documenting (pgdata clearly holds postgres data), easy to reference in backup scripts, and won't be accidentally deleted. Anonymous volumes are invisible unless you know the UUID.

KEY TAKEAWAYS

Named volumes: human-readable names. Easy to manage, share, and backup. Use in production.
Anonymous volumes: UUID names. Tied to container lifecycle. Avoid for important data.
docker volume ls -f dangling=true = find orphaned volumes consuming disk space.
Volume data lives at /var/lib/docker/volumes/<name>/_data on the host.

Q15 How do you share data between multiple containers using volumes? Medium

▸ **DETAILED ANSWER**

Sharing data between containers is a common pattern for sidecar containers, log aggregators, data processing pipelines, and shared configuration. Docker provides clean mechanisms for this.

Method 1 — Shared Named Volume:

```
# Create shared volume:
docker volume create shared-logs

# App container writes logs:
docker run -d \
  --name app \
  -v shared-logs:/var/log/app \
  myapp:latest

# Log aggregator reads same volume:
docker run -d \
  --name log-shipper \
  -v shared-logs:/logs:ro \
  fluentd:latest
# :ro = read-only mount for the log-shipper — it can't write to logs
```

Method 2 — --volumes-from (share all volumes from another container):

```
# Data container with volume:
docker run -d \
  --name data-store \
  -v /app/data \
  busybox

# Multiple containers share ALL volumes from data-store:
docker run -d --name app1 --volumes-from data-store myapp
docker run -d --name app2 --volumes-from data-store myapp

# Classic backup pattern using --volumes-from:
docker run --rm \
  --volumes-from data-store \
  -v $(pwd)/backup:/backup \
  alpine sh -c 'cd /app/data && tar czf /backup/backup.tar.gz .'
```

Method 3 — Docker Compose shared volumes:

```
# docker-compose.yml
version: '3.8'
services:
  app:
    image: myapp:latest
    volumes:
      - shared-data:/app/data

  processor:
```

```
    image: dataprocessor:latest
    volumes:
      - shared-data:/input:ro  # read-only access

volumes:
  shared-data:
```

INTERVIEW TIP

Always mount shared volumes as read-only (:ro) for containers that only need to READ the data. This prevents accidental writes and follows the principle of least privilege. Only the container that produces the data needs write access.

KEY TAKEAWAYS

Named volumes can be mounted into multiple containers simultaneously — they all see the same data.
Use :ro suffix for containers that only read shared data — prevents accidental modification.
--volumes-from mounts ALL volumes from another container. Good for backup sidecar pattern.
Docker Compose makes shared volumes trivial — declare in volumes: section, reference in services.

SECTION 04

Docker Compose

Defining, running and managing multi-container applications

IN THIS SECTION

Q16 What is Docker Compose? How is it different from running individual docker run commands?

Easy

DETAILED ANSWER

Docker Compose is a tool for defining and running MULTI-CONTAINER applications. Instead of running 5 separate docker run commands with long flags and hoping you remember all the options, you define everything in a YAML file and run one command.

Aspect	docker run (manual)	Docker Compose
Startup	Run each container separately	docker-compose up — starts all services
Configuration	Flags on command line (forget easily)	YAML file — version controlled
Networking	Manual network creation and assignment	Auto-creates network, services find each other
Volumes	Specify in each docker run command	Defined once in volumes: section
Reproducibility	Depends on remembering all flags	100% reproducible — git commit the file
Dependencies	Manual ordering by running in sequence	depends_on: for service ordering
Scale	Run manually multiple times	docker-compose up --scale api=3
Teardown	docker rm + docker network rm (multiple steps)	docker-compose down — removes everything

```
# Without Compose — painful:
docker network create myapp-net
docker volume create pgdata
docker run -d --name db --network myapp-net \
  -v pgdata:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=secret \
  -e POSTGRES_DB=myapp postgres:15
```

```
docker run -d --name redis --network myapp-net redis:7
docker run -d --name api --network myapp-net \
  -p 8080:3000 \
  -e DB_HOST=db -e REDIS_HOST=redis myapi:latest

# With Compose — one command:
docker-compose up -d
# All services start, network created, volumes created — done!
```

INTERVIEW TIP

Docker Compose is the standard for local development environments in every DevOps team. A well-written docker-compose.yml means any developer can clone the repo and run the full stack with one command — no setup docs, no manual steps. This is worth its weight in gold for developer productivity.

KEY TAKEAWAYS

Compose = multi-container orchestration for single hosts. One YAML file = entire stack.

docker-compose up -d = start all services in background. docker-compose down = stop + remove.

All services in same compose file are on same custom network automatically.

docker-compose.yml should be committed to git — it IS the infrastructure as code for local dev.

Q17 Explain the structure of a docker-compose.yml file — services, networks, volumes sections.

Medium

DETAILED ANSWER

A docker-compose.yml file has three top-level sections: services (the containers), networks (the connectivity), and volumes (the storage). Understanding each section deeply lets you model complex multi-service architectures cleanly.

```
version: '3.8'   # Compose file format version

# ─── SERVICES (each service = one container) ──────────────────
services:

  postgres-db:
    image: postgres:15-alpine           # use existing image
    container_name: myapp_postgres
    restart: unless-stopped             # auto-restart policy
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: ${DB_PASSWORD} # from .env file
      POSTGRES_DB: myappdb
    volumes:
      - pgdata:/var/lib/postgresql/data # named volume
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql  # bind mount
    networks:
      - backend
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "appuser"]
```

```
      interval: 10s
      timeout: 5s
      retries: 5

  redis-cache:
    image: redis:7-alpine
    networks:
      - backend

  api-service:
    build:                              # build from Dockerfile
      context: ./api
      dockerfile: Dockerfile
      args:
        BUILD_ENV: production
    ports:
      - '8080:3000'                     # host:container
    environment:
      - DB_HOST=postgres-db
      - REDIS_URL=redis://redis-cache:6379
    depends_on:
      postgres-db:
        condition: service_healthy      # wait for healthcheck
    networks:
      - backend
      - frontend
    deploy:
      replicas: 2                       # for Swarm mode
      resources:
        limits:
          cpus: '0.5'
          memory: 512M

  nginx:
    image: nginx:alpine
    ports:
      - '80:80'
      - '443:443'
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    networks:
      - frontend

# ─── NETWORKS ──────────────────────────────────────────
networks:
  backend:
    driver: bridge
  frontend:
    driver: bridge

# ─── VOLUMES ───────────────────────────────────────────
volumes:
  pgdata:
    driver: local
```

INTERVIEW TIP

The 'healthcheck' + 'depends_on: condition: service_healthy' combination is the proper way to handle service startup ordering. This ensures the API doesn't start until Postgres is actually READY to accept connections — not just 'started'. This solves the classic race condition in multi-service apps.

KEY TAKEAWAYS

services: = container definitions. networks: = connectivity. volumes: = storage declarations.
Use 'build: context:' to build from Dockerfile. Use 'image:' to use pre-built image.
healthcheck + depends_on condition: service_healthy = proper startup ordering.
restart: unless-stopped = container auto-restarts on failure but not manual docker stop.

Q18 What is the difference between docker-compose up, down, start, stop, and restart?

Easy

▸ DETAILED ANSWER

Command	What It Does	When to Use
up	CREATE and START containers, networks, volumes	First time or after config changes — full startup
up -d	Same as up but in background (detached)	Standard way to start in production/dev
up --build	Rebuild images THEN start	When Dockerfile or source code changed
down	STOP and REMOVE containers + networks	Clean teardown (volumes kept by default)
down -v	STOP + REMOVE containers, networks, AND volumes	Full cleanup including data — destructive!
start	START existing stopped containers	Resume without recreating
stop	STOP running containers (keep containers)	Pause without removing
restart	Stop then start specific service	Apply config changes to running service
ps	List containers managed by this Compose file	Check current status
logs -f	Follow logs of all services	Live log monitoring
exec	Run command in running service container	Debug running container

```
# Full workflow:
docker-compose up -d              # start all services detached
docker-compose ps                 # check all running
docker-compose logs -f api        # follow api service logs
docker-compose exec api bash      # shell into api container

# Rebuild and restart after code change:
docker-compose up -d --build api  # rebuild only api service

# Scale a service:
docker-compose up -d --scale api=3

# Stop without destroying:
docker-compose stop
docker-compose start              # resume

# Full teardown (keeps volumes):
docker-compose down

# Nuclear option — destroy everything including data:
docker-compose down -v  # WARNING: deletes all volumes!
```

COMMON MISTAKES

docker-compose down -v deletes ALL volumes including your database data. NEVER run in production without backup.

docker-compose up WITHOUT --build uses cached images — code changes won't be reflected. Always use --build after code changes.

docker-compose restart does NOT rebuild images — if config changed, use docker-compose up -d instead.

KEY TAKEAWAYS

up = create + start. down = stop + remove. start/stop = resume/pause without recreating.

up --build = rebuild images first. Always use after Dockerfile or source code changes.

down -v = nuclear option — deletes volumes. Only use when you want to start completely fresh.

docker-compose exec <service> bash = get shell into running service. Most useful debug command.

Q19 How do you handle environment variables in Docker Compose? What is a .env file?

Medium

▸ **DETAILED ANSWER**

Environment variables in Docker Compose are the correct way to handle configuration that varies between environments (dev/staging/prod) without changing the docker-compose.yml file itself.

Multiple ways to set environment variables:

Method	How	Use Case
.env file	Key=value file in same dir as compose file	Default values, local dev secrets

environment: section	Inline in docker-compose.yml	Non-sensitive config that's same across envs
env_file: section	Reference external .env files	Multiple env files for different services
Shell environment	Export in shell before running compose	CI/CD pipeline variables injection
Docker secrets	Swarm-managed encrypted secrets	Production sensitive data (passwords, certs)

```
# .env file (NEVER commit to git if it has secrets):
DB_PASSWORD=supersecret123
DB_USER=appuser
REDIS_PORT=6379
NODE_ENV=development
APP_PORT=8080

# docker-compose.yml references .env automatically:
services:
  api:
    image: myapi:latest
    ports:
      - '${APP_PORT}:3000'  # uses APP_PORT from .env
    environment:
      NODE_ENV: ${NODE_ENV}
      DB_PASSWORD: ${DB_PASSWORD}  # interpolated from .env
      DB_USER: ${DB_USER}
      STATIC_CONFIG: production    # hardcoded — same everywhere

  db:
    image: postgres:15
    env_file:                      # load from separate file
      - ./db.env
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}

# .env.example (COMMIT this — template without real secrets):
# DB_PASSWORD=change_me
# DB_USER=appuser
# NODE_ENV=development
```

COMMON MISTAKES

NEVER commit .env files with real passwords/API keys to git — add .env to .gitignore immediately.

ALWAYS commit .env.example with placeholder values so teammates know what variables are needed.

Environment variables set in docker-compose.yml are visible in 'docker inspect' — don't hardcode secrets there.

INTERVIEW TIP

The professional pattern: .env.example committed to git with placeholder values and comments. .gitignore includes .env. CI/CD pipeline injects real secrets via environment variables or secrets manager (AWS Secrets Manager, Vault). Local devs copy .env.example → .env and fill real values.

✓ KEY TAKEAWAYS

.env file = automatic variable source for docker-compose.yml. Loaded from same directory.
Use ${VARIABLE} syntax in compose file to reference env variables.
NEVER commit .env with secrets. ALWAYS commit .env.example as a template.
For production secrets: use Docker Secrets (Swarm), AWS Secrets Manager, or HashiCorp Vault.

Q20 What is depends_on in Docker Compose? What are its limitations for service readiness?

Hard

▸ **DETAILED ANSWER**

depends_on defines the startup ORDER between services. It prevents a service from starting before its dependencies. But there's a critical limitation that trips up almost every beginner — understanding it separates junior from senior.

The core limitation — 'started' does NOT mean 'ready':

depends_on (basic form) waits for the container to START — not for the application inside it to be READY. PostgreSQL takes 1-3 seconds to initialize after its container starts. If your API starts immediately, it tries to connect to Postgres before Postgres is accepting connections — connection refused.

```
# BAD — depends_on alone is NOT enough:
services:
  api:
    depends_on:
      - postgres      # only waits for container to START
  postgres:
    image: postgres:15
# api starts, postgres container is up but not ready
# api gets: 'connection refused' — race condition!

# GOOD — use healthcheck + condition:
services:
  postgres:
    image: postgres:15
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s

  api:
    depends_on:
      postgres:
        condition: service_healthy  # waits for healthcheck to PASS
    # Now api only starts after pg_isready succeeds

# Other condition options:
# condition: service_started  (default — just container started)
# condition: service_healthy  (healthcheck passed — CORRECT for DBs)
```

```
# condition: service_completed_successfully  (for one-time init jobs)
```

Alternative approach — application-level retry:

Even with healthchecks, the best production-grade applications implement connection retry logic internally. Don't rely 100% on orchestration — build resilience into the app itself.

INTERVIEW TIP

In real projects, use BOTH: healthcheck + condition: service_healthy in docker-compose for startup ordering, AND implement retry logic with exponential backoff in your application code. The orchestration handles initial startup; the retry logic handles transient failures after startup.

KEY TAKEAWAYS

depends_on: service controls startup ORDER but NOT readiness.
Basic depends_on only waits for container to start — NOT for app inside to be ready.
Use healthcheck + condition: service_healthy to wait for actual readiness.
Best practice: healthcheck in Compose + retry logic in app code = truly resilient startup.

SECTION 05

Security & Best Practices

Writing secure, lean, production-ready Docker configurations

IN THIS SECTION

Q21 What are Docker security best practices? How do you run containers as non-root users?

Hard

▸ **DETAILED ANSWER**

Container security is a production-critical concern. By default, processes in Docker containers run as ROOT — a major security risk. If an attacker exploits a vulnerability in your containerized app, they get root access inside the container, which may be enough to escape to the host.

Top Docker Security Practices:

Practice	Implementation
Run as non-root	Add USER instruction in Dockerfile — MOST important practice
Read-only filesystem	docker run --read-only myimage — prevent writes outside volumes
Drop capabilities	docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE nginx
No privileged mode	NEVER use --privileged in production — gives full host access
Limit resources	--memory=512m --cpus=0.5 — prevent resource exhaustion attacks
Use minimal base images	alpine/distroless — smaller attack surface, fewer CVEs
Scan images for CVEs	docker scout, Trivy, Snyk — scan before deploying
Don't store secrets in images	Use environment vars or secrets manager — docker history reveals them
Keep images updated	Regularly rebuild with latest base image patches

Running as non-root — the correct implementation:

```
# Dockerfile — proper non-root setup:
FROM node:20-alpine
```

```
WORKDIR /app

# Create non-root user and group:
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Copy files as root (need write permission for setup):
COPY --chown=appuser:appgroup package*.json ./
RUN npm ci --only=production
COPY --chown=appuser:appgroup . .

# Switch to non-root user for runtime:
USER appuser

# This process now runs as 'appuser' — not root
CMD ["node", "src/index.js"]

# ————————————————————————————————————————
# docker run — additional security flags:
docker run -d \
  --read-only \                    # read-only root filesystem
  --tmpfs /tmp \                   # allow writes only to /tmp
  --cap-drop ALL \                 # drop ALL Linux capabilities
  --cap-add NET_BIND_SERVICE \     # add back only what's needed
  --security-opt no-new-privileges \  # prevent privilege escalation
  --memory=512m \
  --cpus=0.5 \
  myapp:latest
```

INTERVIEW TIP

Running as root in a container is dangerous for two reasons: (1) If the container process is compromised, the attacker has root IN the container — much more powerful. (2) Container escape vulnerabilities (rare but exist) are far more dangerous when the process is already root. Non-root + read-only filesystem + dropped capabilities = defense in depth.

KEY TAKEAWAYS

Containers run as root by default — always add USER <non-root-user> in Dockerfile.
Use --read-only flag to prevent writes to container filesystem (use volumes for writable dirs).
--cap-drop ALL + --cap-add only-what-needed = principle of least privilege for capabilities.
Never use --privileged in production — it gives container access to ALL host devices and capabilities.

Q22 What is the difference between ADD and COPY in Dockerfile? Why is COPY preferred?

Easy

DETAILED ANSWER

Feature	COPY	ADD
Basic file copy	✓ Yes	✓ Yes

Copy directories	✓ Yes	✓ Yes
Extract tar/gzip archives	✗ No — copies as-is	✓ Yes — auto-extracts
Fetch from URL	✗ No	✓ Yes — but insecure!
Transparency	✓ Always does exactly what you expect	✗ Behavior depends on source type
Recommended by Docker	✓ COPY is the official recommendation	□ ADD only when needed
Cache predictability	✓ Highly predictable	□ URL fetches bypass cache

```
# COPY — simple, transparent, predictable (PREFERRED):
COPY ./src /app/src
COPY package.json package-lock.json /app/
COPY --chown=node:node . /app

# ADD — only use for tar extraction:
ADD app.tar.gz /app/   # extracts tar.gz automatically

# ADD from URL — DO NOT USE (security and cache issues):
ADD https://example.com/config.json /app/config.json
# Problems: no cache, checksum not verified, network dependency

# Better alternative to ADD for URL fetching:
RUN curl -fsSL https://example.com/config.json -o /app/config.json
# OR use a separate build stage to download and verify
```

□ INTERVIEW TIP

The Docker documentation itself says: 'COPY is preferred. Use ADD only for tar extraction.' The URL-fetching behavior of ADD is dangerous — it makes your build dependent on an external URL, has no checksum verification, and bypasses the build cache. Use RUN curl if you need to fetch from URLs — it's explicit and controllable.

✓ KEY TAKEAWAYS

COPY = simple, predictable file copying. Always prefer COPY over ADD.
ADD = use ONLY when you need automatic tar.gz extraction into container.
Never use ADD for URL fetching — no checksum, no cache, network dependency.
For URL downloads: use RUN curl/wget with explicit checksum verification instead.

Q23 How do you reduce Docker image size? What is a multi-stage build and when do you use it?

Hard

▸ **DETAILED ANSWER**

Image size matters enormously in production — larger images mean slower CI/CD pipelines, longer deployment times, higher registry storage costs, and larger attack surfaces. Multi-stage builds are the most powerful technique for keeping production images lean.

Image size reduction techniques — ranked by impact:

Technique	Impact	How
Multi-stage builds	□ Highest	Build in fat image, copy artifacts to minimal image
Use Alpine/distroless base	High	node:20-alpine (50MB) vs node:20 (900MB)
Use --no-install-recommends	Medium	apt-get install -y --no-install-recommends pkg
Clean up in same RUN layer	Medium	&& apt-get clean && rm -rf /var/lib/apt/lists/*
Use .dockerignore	Medium	Exclude node_modules, .git, tests from build context
Combine RUN commands	Low-Medium	Fewer layers = potentially smaller image
Use production deps only	Medium	npm ci --only=production (skip devDependencies)

Multi-stage build — the complete example:

```
# ─── Stage 1: Builder (fat image with all build tools) ──────────
FROM golang:1.21 AS builder
# This image is 800MB+ but we don't ship it

WORKDIR /build
COPY go.mod go.sum ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -o myapp ./cmd/server
# Result: a single static binary 'myapp'

# ─── Stage 2: Production image (tiny — just the binary) ─────────
FROM scratch
# 'scratch' = completely empty image. Zero MB base.

COPY --from=builder /build/myapp /myapp
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/

EXPOSE 8080
ENTRYPOINT ["/myapp"]

# Result: ~10MB image vs 800MB+ without multi-stage!
# Contains: ONLY the compiled binary. No Go compiler. No source. No tools.
```

```
# ─── Node.js multi-stage example ──────────────────────────
FROM node:20 AS builder
WORKDIR /build
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build   # compile TypeScript, bundle, etc.

FROM node:20-alpine
WORKDIR /app
COPY --from=builder /build/dist ./dist
COPY --from=builder /build/node_modules ./node_modules
USER node
CMD ["node", "dist/index.js"]
# node:20 = 900MB. node:20-alpine = 50MB. Plus no dev tools in prod.
```

INTERVIEW TIP

Multi-stage builds are especially powerful for compiled languages (Go, Rust, Java, TypeScript). You get the full build toolchain in stage 1, and the final image has ONLY the compiled artifact — no compiler, no source code, no test dependencies. A Go binary that needs 800MB to compile can produce a 10MB final image.

KEY TAKEAWAYS

Multi-stage builds: build in large image, COPY --from=<stage> only artifacts to minimal image.
Use FROM scratch for static binaries (Go, Rust). FROM alpine for apps needing a Linux base.
Alpine base images are ~50MB vs ~500MB for standard images — massive size reduction.
COPY --from=builder selectively copies only what's needed — no build tools in production image.

Q24 What is .dockerignore? How does it work and why is it important? — Easy

DETAILED ANSWER

.dockerignore is a file that tells Docker which files and directories to EXCLUDE from the build context when running docker build. The build context is the set of files sent to the Docker daemon — and without .dockerignore, EVERYTHING in your directory gets sent.

Why .dockerignore matters — the build context problem:

When you run 'docker build .', Docker compresses your ENTIRE current directory and sends it to the Docker daemon. If you have 500MB of node_modules, a large .git history, or big log files — all of that gets sent even if your Dockerfile never COPYs it. This makes builds painfully slow.

```
# .dockerignore — exclude from build context:

# Dependency directories (HUGE — rebuild inside container anyway):
node_modules
.npm
vendor/
```

```
# Version control:
.git
.gitignore
.svn

# Development and test files:
*.test.js
*.spec.ts
__tests__/
coverage/
.nyc_output/

# Build outputs (rebuilt inside container):
dist/
build/
*.tsbuildinfo

# Sensitive files — NEVER send to Docker daemon:
.env
.env.local
.env.*.local
*.pem
*.key
secrets/

# IDE and OS files:
.idea/
.vscode/
.DS_Store
Thumbs.db

# Docker files themselves:
Dockerfile*
docker-compose*

# Logs:
logs/
*.log
npm-debug.log*
```

INTERVIEW TIP

.dockerignore also prevents cache invalidation from irrelevant files. If you COPY . . in your Dockerfile and .git changes with every commit (it always does), your entire build cache is invalidated every time. By ignoring .git, Docker's cache works correctly and only invalidates when actual source files change.

KEY TAKEAWAYS

.dockerignore excludes files from the build context sent to Docker daemon. Think: .gitignore for Docker.
Always exclude: node_modules, .git, .env files, IDE files, logs, dist/build dirs.
Missing .dockerignore = slow builds (huge context) + potential secrets leak in context.
.dockerignore also prevents false cache invalidation — .git changes won't bust your build cache.

SECTION 06

Real-World & Advanced Scenarios

Production debugging, logging, Swarm, and end-to-end troubleshooting

IN THIS SECTION

Q25 What happens when you run `docker run`? Explain the complete internal flow step by step.

Hard

▸ **DETAILED ANSWER**

This question tests whether you understand Docker's internals. Most people know what docker run DOES — few know HOW it works under the hood. This is a favourite senior interview question.

```
docker run -d -p 8080:80 --name webserver nginx:latest

# FULL INTERNAL FLOW:

# STEP 1 — Docker CLI parses the command:
#   Image: nginx:latest  |  Port: 8080->80  |  Name: webserver  |  Detached

# STEP 2 — CLI sends REST API request to Docker Daemon (dockerd):
#   POST /containers/create  {image: nginx, name: webserver, ...}

# STEP 3 — Docker Daemon checks LOCAL image cache:
#   Is nginx:latest already pulled?
#   YES → skip to step 5
#   NO  → step 4

# STEP 4 — Pull image from registry:
#   Docker daemon connects to registry.hub.docker.com
#   Downloads each layer in parallel (content-addressed by SHA256)
#   Stores layers in /var/lib/docker/overlay2/

# STEP 5 — Create container:
#   Docker daemon calls containerd → creates container spec
#   Sets up Union Filesystem: read-only image layers + writable CoW layer
#   Allocates container ID (UUID)

# STEP 6 — Set up networking:
```

```
#   Creates veth pair (virtual ethernet cable)
#   One end goes into container's network namespace
#   Other end connects to docker0 bridge on host
#   Assigns container IP (e.g., 172.17.0.2)
#   Sets up iptables rules for port mapping (8080 → 172.17.0.2:80)

# STEP 7 — Apply resource limits:
#   Creates cgroup for this container
#   Sets CPU/memory limits via cgroup controllers

# STEP 8 — Start the process:
#   containerd calls runc
#   runc uses clone() syscall to create new namespaces:
#     PID namespace, NET namespace, MNT namespace, UTS namespace
#   Starts ENTRYPOINT/CMD as PID 1 inside the container
#   nginx starts, binds to port 80 inside container

# STEP 9 — Container is running:
#   docker run returns container ID
#   -d flag: daemon returns immediately without attaching
#   Port 8080 on host now forwards to port 80 in container
```

INTERVIEW TIP

Knowing this flow lets you debug problems at each step: image pull failures (step 4), networking issues (step 6), resource constraint errors (step 7), process startup failures (step 8). When asked 'what happens in docker run', walk through each step — it demonstrates genuine systems knowledge.

KEY TAKEAWAYS

CLI → REST API → Docker Daemon → containerd → runc → Linux kernel (namespaces + cgroups).
Image layers pulled from registry, stored in /var/lib/docker/overlay2/ by SHA256 hash.
Container gets: veth pair for networking, cgroup for resource limits, isolated namespaces.
ENTRYPOINT/CMD runs as PID 1 inside its own PID namespace — completely isolated process tree.

Q26 How do you debug a failing Docker container? What commands do you use?

Hard

DETAILED ANSWER

Container debugging is a critical production skill. The approach depends on WHETHER the container is running, exited cleanly, or crashed immediately.

Debugging workflow — from the outside in:

```
# ─── STEP 1: Check what's happening ──────────────────────────
docker ps -a                # show ALL containers (including exited)
# STATUS column: Up, Exited (1), Restarting, Created

docker inspect mycontainer   # full JSON config: networking, mounts, env vars, state
```

```
# ── STEP 2: Read the logs ──────────────────────────────────
docker logs mycontainer                  # all logs
docker logs --tail 100 mycontainer       # last 100 lines
docker logs -f mycontainer               # follow (live)
docker logs --since 30m mycontainer      # last 30 minutes
docker logs --timestamps mycontainer     # with timestamps

# ── STEP 3: Get inside running container ──────────────────
docker exec -it mycontainer bash         # bash shell
docker exec -it mycontainer sh           # if bash not available (alpine)
docker exec -it mycontainer ps aux       # check processes inside
docker exec -it mycontainer netstat -tlnp  # check listening ports
docker exec -it mycontainer env          # check environment variables

# ── STEP 4: Container crashed immediately (can't exec) ────
# Override the entrypoint to get a shell instead:
docker run -it --entrypoint bash myimage
docker run -it --entrypoint sh myimage   # for alpine

# ── STEP 5: Check resource usage ──────────────────────────
docker stats mycontainer                 # live CPU/memory/network/disk
docker stats --no-stream                 # one-shot stats

# ── STEP 6: Copy files out for inspection ─────────────────
docker cp mycontainer:/var/log/app.log ./app.log

# ── STEP 7: Check host-level issues ───────────────────────
docker events --since 1h                 # all Docker events
dmesg | grep docker                      # kernel messages about containers
```

INTERVIEW TIP

The most powerful debug trick: if a container crashes immediately (can't exec into it), override the ENTRYPOINT with bash: 'docker run -it --entrypoint bash myimage'. Now you're inside the image's filesystem without the app running — you can manually run the start command and see exactly what error it produces.

KEY TAKEAWAYS

docker logs = first command always. docker inspect = full config dump for environment/mount issues.
docker exec -it <c> bash = shell inside running container for live investigation.
Container crashes immediately → docker run -it --entrypoint bash myimage to debug.
docker stats = live resource usage. Check for OOM kills if container keeps restarting.

Q27 What is the difference between docker stop and docker kill? Which is safer and why?

Easy

▶ DETAILED ANSWER

Aspect	docker stop	docker kill
Signal sent	SIGTERM (graceful shutdown request)	SIGKILL (immediate forced termination)

Process behavior	Application handles signal — can clean up, flush buffers, close connections	Process killed INSTANTLY — no cleanup possible
Grace period	10 second timeout, then SIGKILL automatically	No timeout — instant death
Data safety	Safe — application completes in-flight work	Risky — may corrupt files, leave locks, lose data
Use case	Normal shutdown — ALWAYS try this first	Last resort — process is hanging and won't stop
Custom signal	docker stop (always SIGTERM)	docker kill -s SIGTERM (can specify any signal)

```
# ALWAYS start with docker stop:
docker stop mycontainer             # sends SIGTERM, waits 10s, then SIGKILL
docker stop -t 30 mycontainer       # give 30 seconds for graceful shutdown

# What the app should do on SIGTERM:
# 1. Stop accepting new requests
# 2. Finish processing in-flight requests
# 3. Flush write buffers / commit database transactions
# 4. Release file locks and close connections
# 5. Exit cleanly with code 0

# Only use kill when stop doesn't work:
docker kill mycontainer             # SIGKILL — instant death

# Sending other signals:
docker kill -s SIGUSR1 mycontainer  # reload config without restart (nginx)
docker kill -s SIGHUP mycontainer   # hangup — many daemons use this to reload

# Check exit code after stop:
docker inspect mycontainer | grep ExitCode
# ExitCode 0 = clean. ExitCode 137 = killed (128+9). ExitCode 1 = error
```

INTERVIEW TIP

In production, configure a sensible grace period with 'docker stop -t 60' for services that handle long requests (file uploads, video processing). The default 10 seconds may cut off legitimate long-running operations. Services should also implement SIGTERM handlers to do cleanup work before exiting.

KEY TAKEAWAYS

docker stop = SIGTERM first (graceful). Waits 10s, then SIGKILL. ALWAYS try this first.
docker kill = SIGKILL immediately. No cleanup. Last resort for hanging containers.
Exit code 137 = process was SIGKILL'd (128 + signal 9). Investigate why it wouldn't stop.
Use exec form ["cmd"] in Dockerfile so process is PID 1 and receives signals directly.

Q28 How do you handle container logs in production? What is the difference between log drivers?

Medium

▶ DETAILED ANSWER

In production, container logs need to be collected, shipped to a centralized system, and retained for debugging and compliance. The default Docker logging is fine for development but insufficient for production.

Log Driver	Where Logs Go	Use Case
json-file (default)	Files in /var/lib/docker/containers/	Development. Local docker logs command works.
journald	systemd journal	When host uses systemd. Integrates with journalctl.
syslog	System syslog	Legacy systems, centralized syslog servers.
fluentd	Fluentd daemon	EFK stack (Elasticsearch + Fluentd + Kibana).
awslogs	AWS CloudWatch Logs	AWS ECS/EC2 deployments.
splunk	Splunk HTTP Event Collector	Enterprise Splunk deployments.
gelf	Graylog	GELF-compatible log systems.
none	Discards all logs	Containers that should produce no logs.

```
# Set log driver per container:
docker run -d \
  --log-driver awslogs \
  --log-opt awslogs-group=/prod/myapp \
  --log-opt awslogs-region=us-east-1 \
  --log-opt awslogs-stream=api-service \
  myapp:latest

# Limit log file size (CRITICAL for production disk management):
docker run -d \
  --log-driver json-file \
  --log-opt max-size=50m \     # each log file max 50MB
  --log-opt max-file=5 \       # keep 5 rotated files = max 250MB
  myapp:latest

# Set default log driver in /etc/docker/daemon.json:
# {
#   "log-driver": "json-file",
#   "log-opts": {
#     "max-size": "50m",
#     "max-file": "5"
#   }
# }

# View logs even when using non-json-file driver:
docker logs mycontainer   # only works with json-file and journald drivers
```

COMMON MISTAKES

Default json-file driver with no size limits fills up disk in production — containers can generate GBs of logs.

Always set --log-opt max-size and max-file to prevent disk exhaustion from runaway logging.

When using non-json-file drivers (awslogs, fluentd), docker logs command no longer works — logs only in the external system.

✓ KEY TAKEAWAYS

json-file (default) = logs on host disk. Always set max-size and max-file limits.
awslogs = for AWS. fluentd = for EFK stack. journald = for systemd hosts.
When using external log drivers, docker logs stops working — plan your debugging workflow.
Best practice: structured JSON logging from app + centralized log aggregation (EFK/ELK/CloudWatch).

Q29 What is Docker Swarm? How is it different from Kubernetes? — Medium

▸ **DETAILED ANSWER**

Aspect	Docker Swarm	Kubernetes (K8s)
Complexity	Simple — built into Docker	Complex — significant learning curve
Setup time	Minutes (docker swarm init)	Hours/days for production setup
Scaling	docker service scale api=5	kubectl scale deployment api --replicas=5
Load balancing	Built-in routing mesh	Needs Ingress controller configuration
Auto-healing	Yes — restarts failed containers	Yes — more sophisticated health management
Rolling updates	Built-in with docker service update	Rolling updates with deployment strategies
Storage	Basic volume support	Rich storage classes, StatefulSets, PVCs
Networking	Overlay networks	CNI plugins (Calico, Flannel, Cilium)
Ecosystem	Smaller, limited tooling	Massive ecosystem (Helm, Istio, Argo etc.)
Production adoption	Declining — legacy use	Industry standard for container orchestration
Best for	Simple multi-host setups, small teams	Enterprise production, complex microservices

```
# Docker Swarm — basic workflow:
docker swarm init                          # initialize manager node
docker swarm join --token <token> <ip>     # worker joins swarm

# Deploy a service (replicated across nodes):
docker service create \
  --name api \
  --replicas 3 \
  --publish 8080:3000 \
  --network myoverlay \
  myapi:latest

# Scale service:
docker service scale api=5

# Rolling update:
```

```
docker service update --image myapi:v2 api

# Check service status:
docker service ls
docker service ps api
```

INTERVIEW TIP

In interviews, acknowledge that Swarm is simpler but Kubernetes is the production industry standard. The honest answer: Swarm is great for smaller teams who need basic orchestration without Kubernetes' complexity. But for most modern production environments, Kubernetes is the choice. Understanding Swarm deeply actually helps you understand Kubernetes concepts faster.

KEY TAKEAWAYS

Docker Swarm = built-in orchestration. Simple, fast setup. Good for small-medium deployments.
Kubernetes = industry standard. Complex but powerful. Standard for modern production.
Both provide: service scaling, rolling updates, auto-healing, load balancing.
Career advice: learn Docker Swarm concepts, but invest deeply in Kubernetes for production jobs.

Q30 A container is running but the application inside is not responding. How do you troubleshoot end-to-end?

Hard

▸ **DETAILED ANSWER**

This is the ultimate real-world Docker troubleshooting question. The container shows as 'Up' but the app is not working. This requires systematic investigation at multiple layers.

Step-by-step investigation:

```
# ── PHASE 1: Verify the problem ──────────────────────────
docker ps                          # confirm container is actually 'Up'
curl -v http://localhost:8080/health # test the endpoint directly
# → Connection refused? → Port issue (step 2)
# → Connection timeout? → App hung (step 3)
# → HTTP 500? → Application error (step 4)

# ── PHASE 2: Check port mapping ──────────────────────────
docker port mycontainer            # verify port mapping exists
ss -tlnp | grep 8080               # is host actually listening on 8080?
docker inspect mycontainer | grep -A 10 'PortBindings'

# ── PHASE 3: Check resource usage ────────────────────────
docker stats mycontainer           # CPU/memory — is app maxed out?
# Memory at 100% → OOM (Out of Memory) — app killed internally
# CPU at 100% → deadlock or infinite loop

# ── PHASE 4: Read application logs ───────────────────────
docker logs --tail 200 mycontainer
docker logs --since 30m mycontainer
# Look for: OOM errors, exceptions, panic, connection errors, startup failure
```

```
# ── PHASE 5: Inspect running processes inside container ──────────
docker exec mycontainer ps aux      # is the app process actually running?
docker exec mycontainer top         # live process CPU/memory inside container
# App process missing? → crashed silently (check logs again)

# ── PHASE 6: Check app is listening on right port ────────────────
docker exec mycontainer ss -tlnp
# Is the app actually bound to the right port inside container?
# App bound to 127.0.0.1:3000 instead of 0.0.0.0:3000?
# → Fix: configure app to bind to 0.0.0.0, not localhost

# ── PHASE 7: Check network connectivity ──────────────────────────
docker exec mycontainer curl http://localhost:3000/health
# Works from inside? → Docker port mapping issue
# Fails from inside? → App not listening or wrong port

# ── PHASE 8: Check dependencies ──────────────────────────────────
docker exec mycontainer curl http://postgres-db:5432  # can reach DB?
docker exec mycontainer env | grep DB_HOST            # correct DB config?

# ── PHASE 9: Check host network/firewall ─────────────────────────
sudo iptables -L DOCKER -n         # check Docker iptables rules
docker network inspect bridge       # verify container IP on bridge
```

Most common root causes:

Symptom	Root Cause	Fix
Connection refused on host port	App not binding to 0.0.0.0 inside container	Configure app to bind 0.0.0.0 not 127.0.0.1
HTTP 500 errors	App error — database connection, missing config	Check logs, fix config/env vars
Container restarts frequently	OOM kill or app crash	Increase memory limit, fix memory leak
Slow responses / timeouts	Resource exhaustion or dependency slow	docker stats, check DB/external service latency
App process not running	Crashed silently — wrong CMD/ENTRYPOINT	docker logs, override ENTRYPOINT to debug

INTERVIEW TIP

The single most common cause of 'container up but app unreachable': the app binds to 127.0.0.1 (localhost) inside the container. Port mapping forwards external traffic to the container's network interface — but if the app only listens on loopback (127.0.0.1), the forwarded traffic arrives at the right IP but the app isn't listening there. Fix: bind to 0.0.0.0.

KEY TAKEAWAYS

Phase 1: Verify HOW it fails (refused vs timeout vs 500) — determines the investigation path.

Phase 3: docker stats — OOM and CPU maxed are silent killers that look like 'app not responding'.

> Phase 6: Check if app is binding to 0.0.0.0 vs 127.0.0.1 inside container — most common issue.
> Phase 4: docker logs is always in the investigation. The app almost always tells you what's wrong.

APPENDIX

Quick Reference — All 30 Questions

Q01	Easy	Docker vs VM — architecture, namespaces, cgroups
Q02	Easy	Docker Image vs Container — relationship, CoW layer
Q03	Medium	Dockerfile — FROM, RUN, COPY, CMD, ENTRYPOINT, ENV, EXPOSE, WORKDIR
Q04	Hard	CMD vs ENTRYPOINT — exec form vs shell form, signal handling
Q05	Easy	Docker Hub vs private registries — naming conventions
Q06	Hard	Docker layers, layer caching, build optimization strategies
Q07	Medium	Network types — bridge, host, none, overlay, macvlan
Q08	Medium	Container communication — --link (legacy) vs custom networks
Q09	Easy	Port mapping — -p vs -P, EXPOSE vs publish, 0.0.0.0 binding
Q10	Medium	Docker DNS — how containers resolve each other by name
Q11	Hard	Overlay networks — VXLAN, Swarm multi-host networking
Q12	Medium	Volume vs Bind Mount vs tmpfs — when to use each
Q13	Easy	Data persistence — what happens when container is removed?
Q14	Medium	Named vs anonymous volumes — docker volume commands
Q15	Medium	Sharing data between containers — volumes-from pattern
Q16	Easy	Docker Compose vs docker run — why Compose wins
Q17	Medium	docker-compose.yml structure — services, networks, volumes
Q18	Easy	up vs down vs start vs stop vs restart — full command guide
Q19	Medium	Environment variables — .env files, interpolation, secrets
Q20	Hard	depends_on — limitation, healthcheck + condition: service_healthy
Q21	Hard	Security best practices — non-root, read-only, cap-drop
Q22	Easy	ADD vs COPY — why COPY is always preferred
Q23	Hard	Image size reduction — multi-stage builds deep dive
Q24	Easy	.dockerignore — build context, cache optimization, security
Q25	Hard	docker run internals — complete flow: CLI → daemon → containerd → runc
Q26	Hard	Debugging failing containers — exec, logs, inspect, override entrypoint
Q27	Easy	docker stop vs docker kill — SIGTERM vs SIGKILL
Q28	Medium	Container logging — log drivers, max-size limits, production strategy

Q29	Medium	Docker Swarm vs Kubernetes — when to use which
Q30	Hard	Container up but app not responding — end-to-end troubleshoot

---

DevOps Interview Series by Sandhya

Linux & Shell ✓ • Docker ✓ • Kubernetes • CI/CD • Terraform • Cloud

Follow on LinkedIn — Next: Kubernetes